

```

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

import joblib

import warnings

warnings.filterwarnings("ignore")

df = pd.read_csv("energydata_complete.csv")
df.head()

```

	date	Appliances	lights	T1	RH_1	T2
RH_2 \						
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.244.790000
1	2016-01-11 17:10:00	60	30	19.89	46.693333	19.244.722500
2	2016-01-11 17:20:00	50	30	19.89	46.300000	19.244.626667
3	2016-01-11 17:30:00	50	40	19.89	46.066667	19.244.590000
4	2016-01-11 17:40:00	60	40	19.89	46.333333	19.244.530000

	T3	RH_3	T4	...	T9	RH_9	T_out
Press_mm_hg \							
0	19.79733.5	44.730000	19.000000	...	17.033333	45.53	6.600000
1	19.79733.6	44.790000	19.000000	...	17.066667	45.56	6.483333
2	19.79733.7	44.933333	18.926667	...	17.000000	45.50	6.366667
3	19.79733.8	45.000000	18.890000	...	17.000000	45.40	6.250000

```
4 19.79 45.000000 18.890000 ... 17.000000 45.40 6.133333
733.9
```

	RH_out	Windspeed	Visibility	Tdewpoint	rv1	rv2
0	92.0	7.000000	63.000000	5.3	13.275433	13.275433
1	92.0	6.666667	59.166667	5.2	18.606195	18.606195
2	92.0	6.333333	55.333333	5.1	28.642668	28.642668
3	92.0	6.000000	51.500000	5.0	45.410389	45.410389
4	92.0	5.666667	47.666667	4.9	10.084097	10.084097

```
[5 rows x 29 columns]
```

```
print("Rows :",df.shape[0])
```

```
print("Columns :",df.shape[1])
```

```
Rows : 19735
```

```
Columns : 29
```

```
df.columns
```

```
Index(['date', 'Appliances', 'lights', 'T1', 'RH_1', 'T2', 'RH_2',
      'T3',
      'RH_3', 'T4', 'RH_4', 'T5', 'RH_5', 'T6', 'RH_6', 'T7', 'RH_7',
      'T8',
      'RH_8', 'T9', 'RH_9', 'T_out', 'Press_mm_hg', 'RH_out',
      'Windspeed',
      'Visibility', 'Tdewpoint', 'rv1', 'rv2'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 19735 entries, 0 to 19734
```

```
Data columns (total 29 columns):
```

#	Column	Non-Null Count	Dtype
0	date	19735 non-null	object
1	Appliances	19735 non-null	int64
2	lights	19735 non-null	int64
3	T1	19735 non-null	float64
4	RH_1	19735 non-null	float64
5	T2	19735 non-null	float64
6	RH_2	19735 non-null	float64
7	T3	19735 non-null	float64
8	RH_3	19735 non-null	float64
9	T4	19735 non-null	float64
10	RH_4	19735 non-null	float64
11	T5	19735 non-null	float64
12	RH_5	19735 non-null	float64
13	T6	19735 non-null	float64

```
14 RH_6      19735 non-null float64
15 T7        19735 non-null float64
16 RH_7      19735 non-null float64
17 T8        19735 non-null float64
18 RH_8      19735 non-null float64
19 T9        19735 non-null float64
20 RH_9      19735 non-null float64
21 T_out     19735 non-null float64
22 Press_mm_hg 19735 non-null float64
23 RH_out     19735 non-null float64
24 Windspeed  19735 non-null float64
25 Visibility 19735 non-null float64
26 Tdewpoint  19735 non-null float64
27 rv1        19735 non-null float64
28 rv2        19735 non-null float64
dtypes: float64(26), int64(2), object(1)
memory usage: 4.4+ MB
```

```
df.isnull().sum()
```

```
date      0
Appliances 0
lights    0
T1         0
RH_1       0
T2         0
RH_2       0
T3         0
RH_3       0
T4         0
RH_4       0
T5         0
RH_5       0
T6         0
RH_6       0
T7         0
RH_7       0
T8         0
RH_8       0
T9         0
RH_9       0
T_out     0
Press_mm_hg 0
RH_out     0
Windspeed  0
Visibility 0
Tdewpoint  0
rv1        0
rv2        0
dtype: int64
```

```
print("Duplicate Rows :",df.duplicated().sum())
```

```
Duplicate Rows : 0
```

```
df.describe()
```

	Appliances	lights	T1	RH_1
T2 \				
count	19735.000000	19735.000000	19735.000000	19735.000000
mean	97.694958	3.801875	21.686571	40.259739
std	102.524891	7.935988	1.606066	3.979299
min	10.000000	0.000000	16.790000	27.023333
25%	50.000000	0.000000	20.760000	37.333333
50%	60.000000	0.000000	21.600000	39.656667
75%	100.000000	0.000000	22.600000	43.066667
max	1080.000000	70.000000	26.260000	63.360000

	RH_2	T3	RH_3	T4
RH_4 \				
count	19735.000000	19735.000000	19735.000000	19735.000000
mean	40.420420	22.267611	39.242500	20.855335
std	4.069813	2.006111	3.254576	2.042884
min	20.463333	17.200000	28.766667	15.100000
25%	37.900000	20.790000	36.900000	19.530000
50%	40.500000	22.100000	38.530000	20.666667
75%	43.260000	23.290000	41.760000	22.100000
max	56.026667	29.236000	50.163333	26.200000

	...	T9	RH_9	T_out	Press_mm_hg \
count	...	19735.000000	19735.000000	19735.000000	19735.000000
mean	...	19.485828	41.552401	7.411665	755.522602
std	...	2.014712	4.151497	5.317409	7.399441
min	...	14.890000	29.166667	-5.000000	729.300000
25%	...	18.000000	38.500000	3.666667	750.933333

50%	...	19.390000	40.900000	6.916667	756.100000
75%	...	20.600000	44.338095	10.408333	760.933333
max	...	24.500000	53.326667	26.100000	772.300000

	RH_out	Windspeed	Visibility	Tdewpoint
rv1 \				
count	19735.000000	19735.000000	19735.000000	19735.000000
mean	79.750418	4.039752	38.330834	3.760707
std	14.901088	2.451221	11.794719	4.194648
min	24.000000	0.000000	1.000000	-6.600000
25%	70.333333	2.000000	29.000000	0.900000
50%	83.666667	3.666667	40.000000	3.433333
75%	91.666667	5.500000	40.000000	6.566667
max	100.000000	14.000000	66.000000	15.500000

	rv2
count	19735.000000
mean	24.988033
std	14.496634
min	0.005322
25%	12.497889
50%	24.897653
75%	37.583769
max	49.996530

[8 rows x 28 columns]

```
df["date"] = pd.to_datetime(df["date"])
```

```
df["Year"] = df["date"].dt.year
```

```
df["Month"] = df["date"].dt.month
```

```
df["Day"] = df["date"].dt.day
```

```
df["Hour"] = df["date"].dt.hour
```

```
df.head()
```

	date	Appliances	lights	T1	RH_1	T2
RH_2 \						
0	2016-01-11 17:00:00	60	30	19.89	47.596667	19.2

1	2016-01-11 17:10:00	60	30	19.89	46.693333	19.2
	44.722500					
2	2016-01-11 17:20:00	50	30	19.89	46.300000	19.2
	44.626667					
3	2016-01-11 17:30:00	50	40	19.89	46.066667	19.2
	44.590000					
4	2016-01-11 17:40:00	60	40	19.89	46.333333	19.2
	44.530000					

	T3	RH_3	T4	...	RH_out	Windspeed	Visibility
Tdewpoint \							
0	19.79	44.730000	19.000000	...	92.0	7.000000	63.000000
5.3							
1	19.79	44.790000	19.000000	...	92.0	6.666667	59.166667
5.2							
2	19.79	44.933333	18.926667	...	92.0	6.333333	55.333333
5.1							
3	19.79	45.000000	18.890000	...	92.0	6.000000	51.500000
5.0							
4	19.79	45.000000	18.890000	...	92.0	5.666667	47.666667
4.9							

	rv1	rv2	Year	Month	Day	Hour
0	13.275433	13.275433	2016	1	11	17
1	18.606195	18.606195	2016	1	11	17
2	28.642668	28.642668	2016	1	11	17
3	45.410389	45.410389	2016	1	11	17
4	10.084097	10.084097	2016	1	11	17

[5 rows x 33 columns]

```
df.to_csv("cleaned_energy_data.csv",index=False)
```

```
print("="*50)
```

```
print("DATASET SHAPE")
```

```
print("="*50)
```

```
print("Rows :",df.shape[0])
```

```
print("Columns :",df.shape[1])
```

```
=====
```

```
DATASET SHAPE
```

```
=====
```

```
Rows : 19735
```

```
Columns : 33
```

```
print("="*50)
```

```
print("DATA TYPES")
```

```
print("="*50)
```

```
df.dtypes
```

```
=====
DATA TYPES
=====
```

```
date                datetime64[ns]
Appliances           int64
lights              int64
T1                  float64
RH_1                float64
T2                  float64
RH_2                float64
T3                  float64
RH_3                float64
T4                  float64
RH_4                float64
T5                  float64
RH_5                float64
T6                  float64
RH_6                float64
T7                  float64
RH_7                float64
T8                  float64
RH_8                float64
T9                  float64
RH_9                float64
T_out               float64
Press_mm_hg         float64
RH_out              float64
Windspeed           float64
Visibility           float64
Tdewpoint           float64
rv1                 float64
rv2                 float64
Year                int64
Month               int64
Day                 int64
Hour                int64
dtype: object
```

```
print("="*50)
print("MISSING VALUES")
print("="*50)
```

```
df.isnull().sum()
```

```
=====
MISSING VALUES
=====
```

```

date          0
Appliances    0
lights        0
T1            0
RH_1          0
T2            0
RH_2          0
T3            0
RH_3          0
T4            0
RH_4          0
T5            0
RH_5          0
T6            0
RH_6          0
T7            0
RH_7          0
T8            0
RH_8          0
T9            0
RH_9          0
T_out         0
Press_mm_hg   0
RH_out        0
Windspeed     0
Visibility    0
Tdewpoint     0
rv1           0
rv2           0
Year          0
Month         0
Day           0
Hour          0
dtype: int64

print("="*50)
print("DUPLICATE ROWS")
print("="*50)

print(df.duplicated().sum())

=====
DUPLICATE ROWS
=====
0

df.describe().T

```

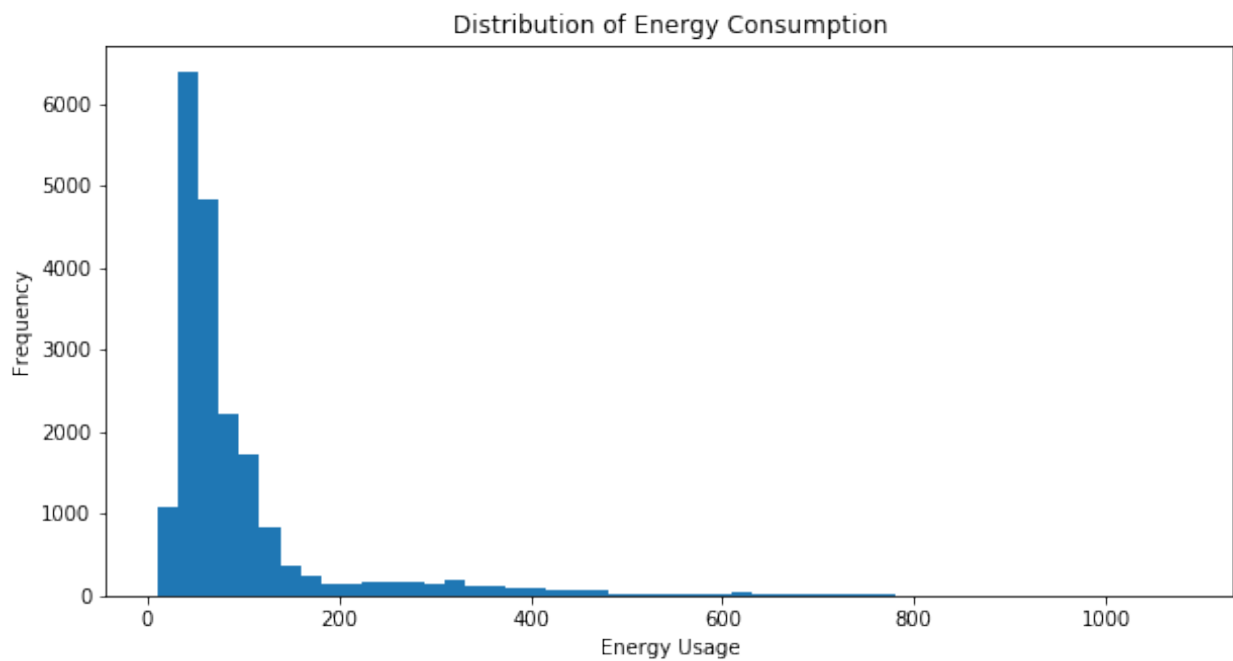
	count	mean	std	min
25%	\			

Appliances	19735.0	97.694958	102.524891	10.000000
50.000000				
lights	19735.0	3.801875	7.935988	0.000000
0.000000				
T1	19735.0	21.686571	1.606066	16.790000
20.760000				
RH_1	19735.0	40.259739	3.979299	27.023333
37.333333				
T2	19735.0	20.341219	2.192974	16.100000
18.790000				
RH_2	19735.0	40.420420	4.069813	20.463333
37.900000				
T3	19735.0	22.267611	2.006111	17.200000
20.790000				
RH_3	19735.0	39.242500	3.254576	28.766667
36.900000				
T4	19735.0	20.855335	2.042884	15.100000
19.530000				
RH_4	19735.0	39.026904	4.341321	27.660000
35.530000				
T5	19735.0	19.592106	1.844623	15.330000
18.277500				
RH_5	19735.0	50.949283	9.022034	29.815000
45.400000				
T6	19735.0	7.910939	6.090347	-6.065000
3.626667				
RH_6	19735.0	54.609083	31.149806	1.000000
30.025000				
T7	19735.0	20.267106	2.109993	15.390000
18.700000				
RH_7	19735.0	35.388200	5.114208	23.200000
31.500000				
T8	19735.0	22.029107	1.956162	16.306667
20.790000				
RH_8	19735.0	42.936165	5.224361	29.600000
39.066667				
T9	19735.0	19.485828	2.014712	14.890000
18.000000				
RH_9	19735.0	41.552401	4.151497	29.166667
38.500000				
T_out	19735.0	7.411665	5.317409	-5.000000
3.666667				
Press_mm_hg	19735.0	755.522602	7.399441	729.300000
750.933333				
RH_out	19735.0	79.750418	14.901088	24.000000
70.333333				
Windspeed	19735.0	4.039752	2.451221	0.000000
2.000000				
Visibility	19735.0	38.330834	11.794719	1.000000

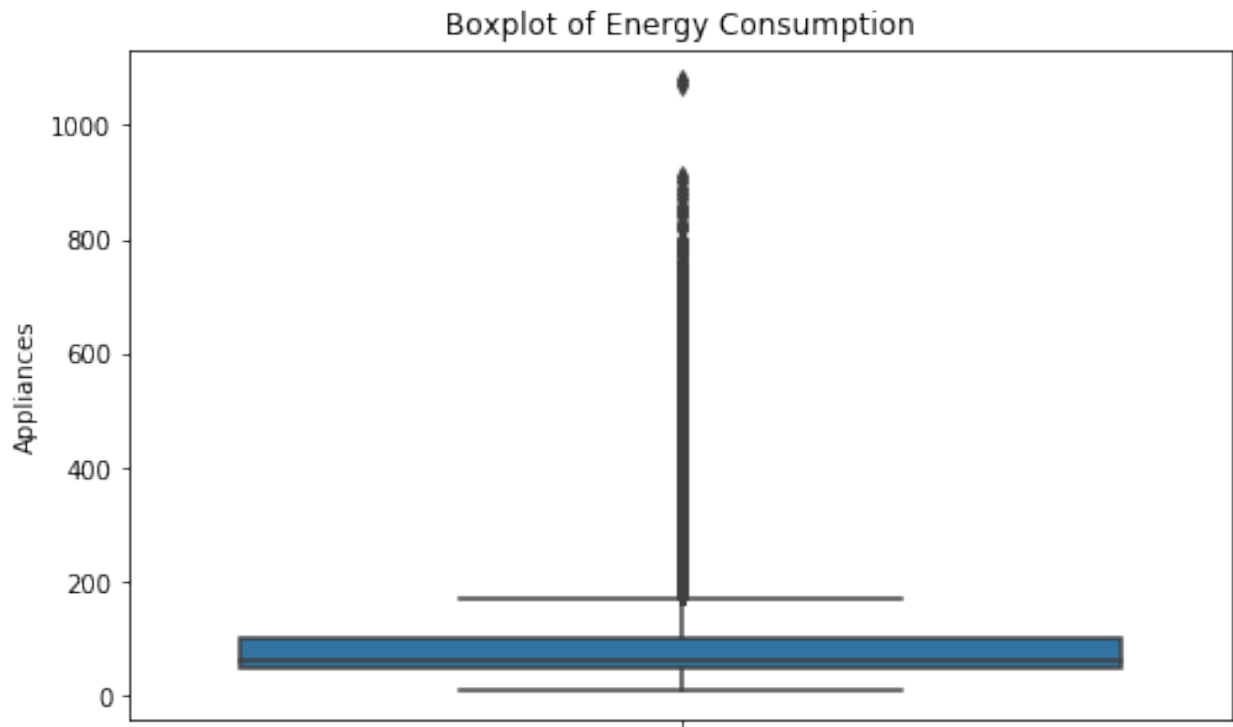
29.000000				
Tdewpoint	19735.0	3.760707	4.194648	-6.600000
0.900000				
rv1	19735.0	24.988033	14.496634	0.005322
12.497889				
rv2	19735.0	24.988033	14.496634	0.005322
12.497889				
Year	19735.0	2016.000000	0.000000	2016.000000
2016.000000				
Month	19735.0	3.101647	1.339200	1.000000
2.000000				
Day	19735.0	16.057411	8.450998	1.000000
9.000000				
Hour	19735.0	11.502002	6.921953	0.000000
6.000000				

	50%	75%	max
Appliances	60.000000	100.000000	1080.000000
lights	0.000000	0.000000	70.000000
T1	21.600000	22.600000	26.260000
RH_1	39.656667	43.066667	63.360000
T2	20.000000	21.500000	29.856667
RH_2	40.500000	43.260000	56.026667
T3	22.100000	23.290000	29.236000
RH_3	38.530000	41.760000	50.163333
T4	20.666667	22.100000	26.200000
RH_4	38.400000	42.156667	51.090000
T5	19.390000	20.619643	25.795000
RH_5	49.090000	53.663333	96.321667
T6	7.300000	11.256000	28.290000
RH_6	55.290000	83.226667	99.900000
T7	20.033333	21.600000	26.000000
RH_7	34.863333	39.000000	51.400000
T8	22.100000	23.390000	27.230000
RH_8	42.375000	46.536000	58.780000
T9	19.390000	20.600000	24.500000
RH_9	40.900000	44.338095	53.326667
T_out	6.916667	10.408333	26.100000
Press_mm_hg	756.100000	760.933333	772.300000
RH_out	83.666667	91.666667	100.000000
Windspeed	3.666667	5.500000	14.000000
Visibility	40.000000	40.000000	66.000000
Tdewpoint	3.433333	6.566667	15.500000
rv1	24.897653	37.583769	49.996530
rv2	24.897653	37.583769	49.996530
Year	2016.000000	2016.000000	2016.000000
Month	3.000000	4.000000	5.000000
Day	16.000000	23.000000	31.000000
Hour	12.000000	17.000000	23.000000

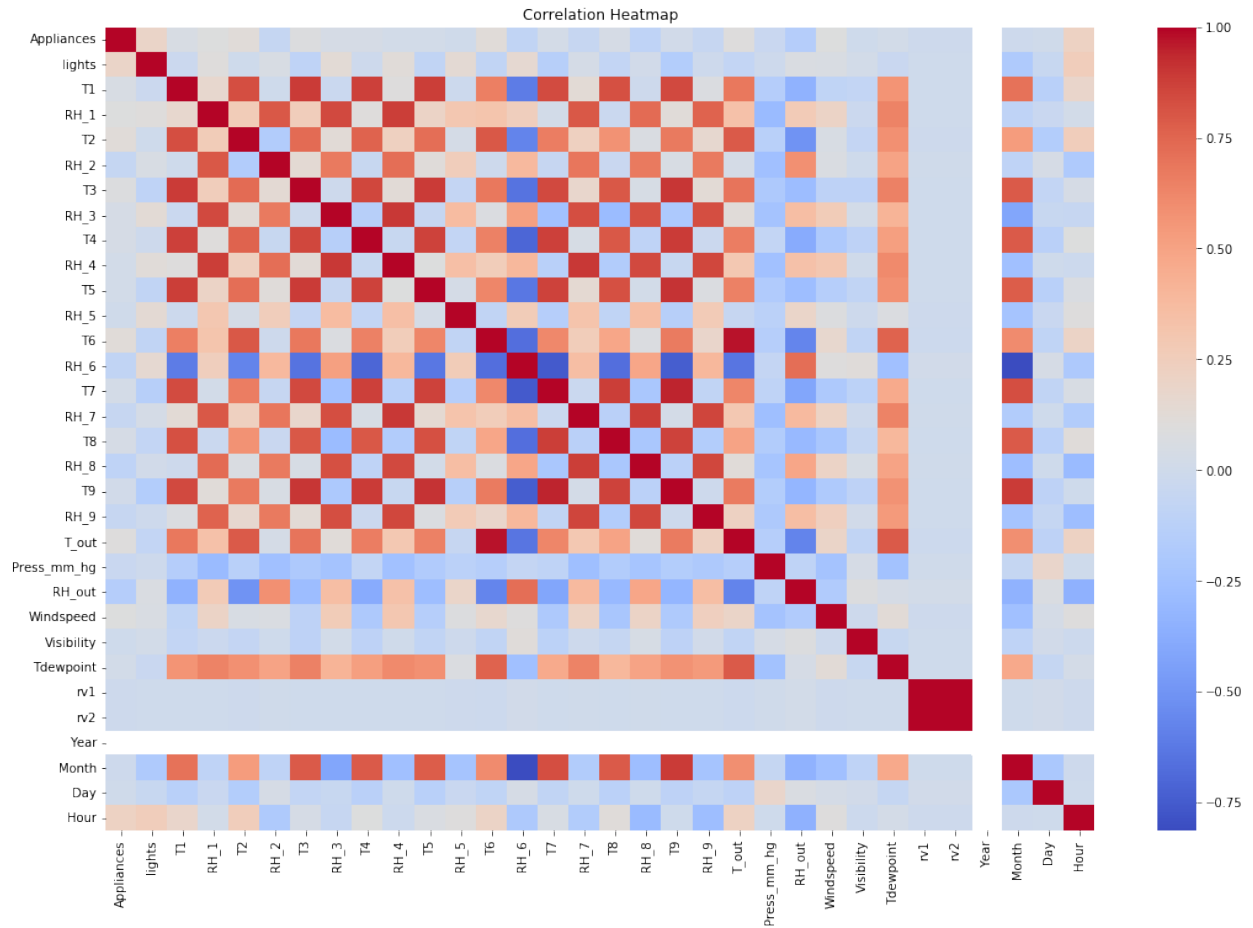
```
plt.figure(figsize=(10,5))
plt.hist(df["Appliances"], bins=50)
plt.title("Distribution of Energy Consumption")
plt.xlabel("Energy Usage")
plt.ylabel("Frequency")
plt.show()
```



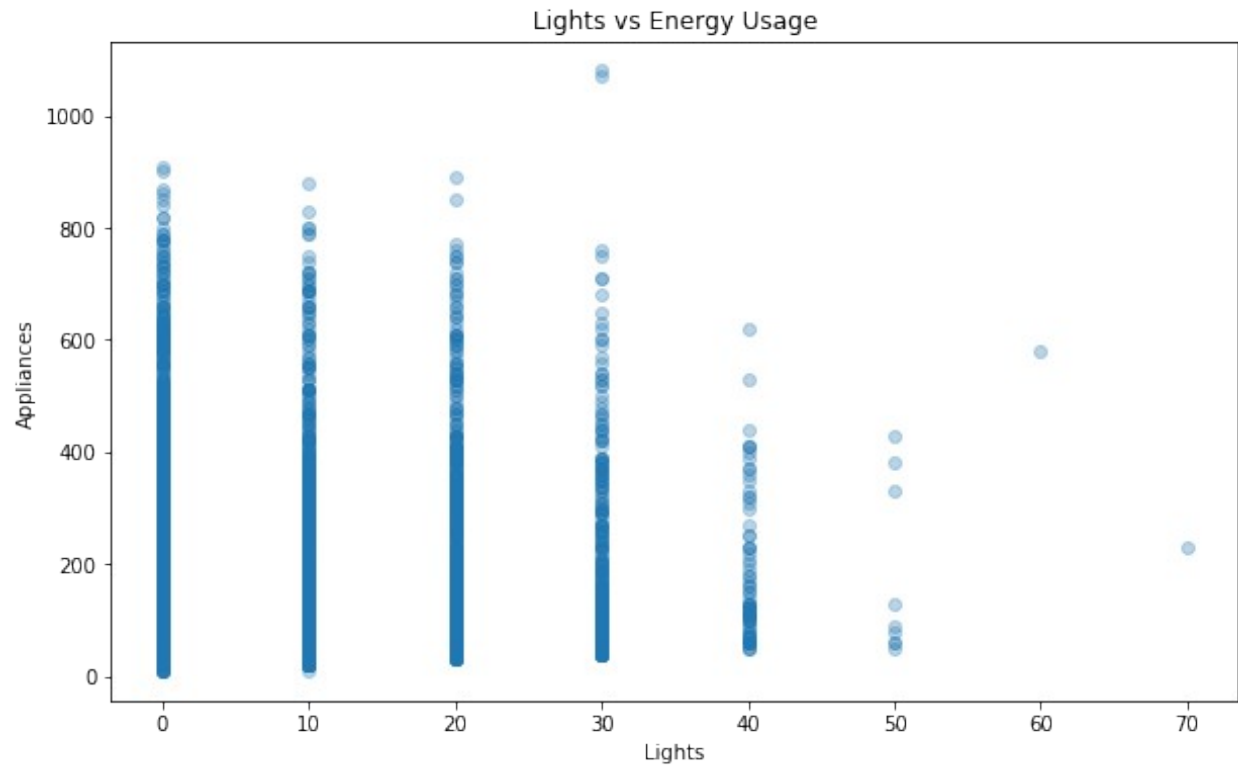
```
plt.figure(figsize=(8,5))
sns.boxplot(y=df["Appliances"])
plt.title("Boxplot of Energy Consumption")
plt.show()
```



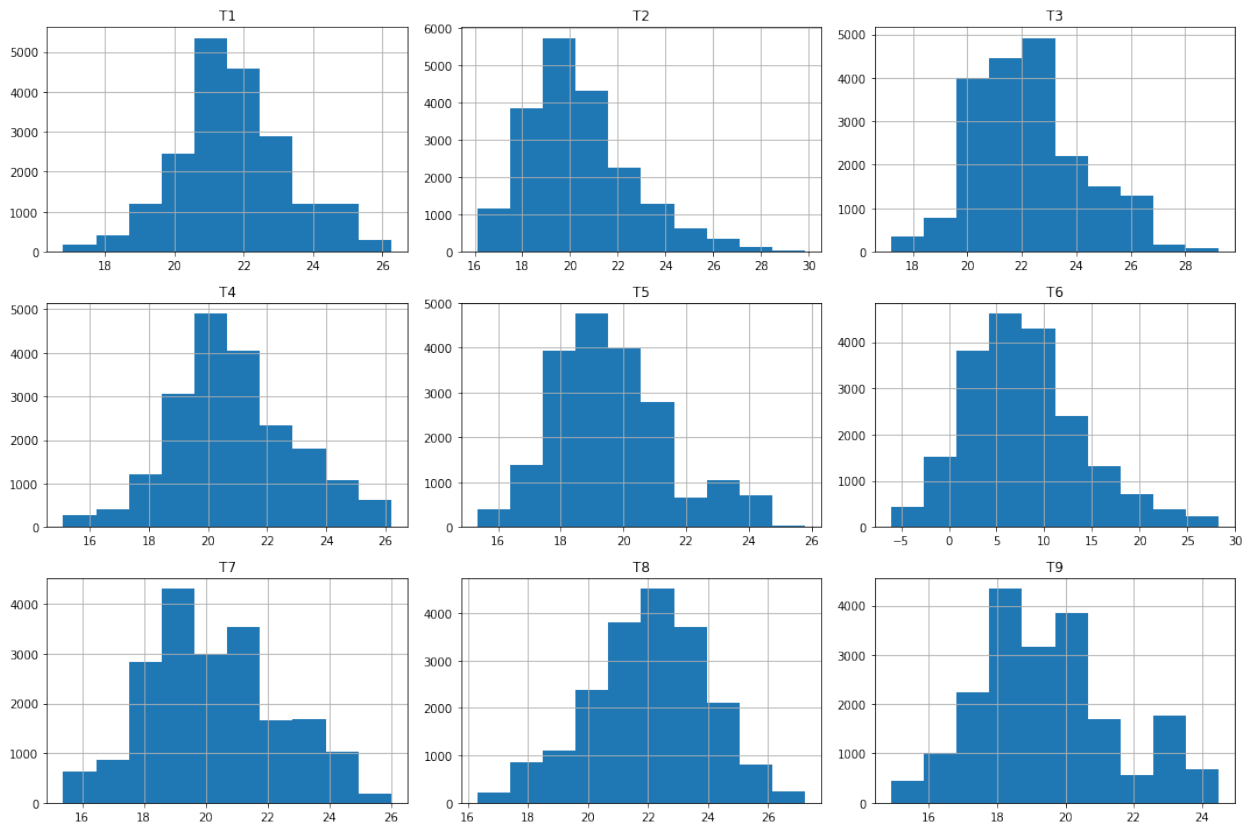
```
import numpy as np
plt.figure(figsize=(18,12))
numeric_df = df.select_dtypes(include=[np.number])
corr = numeric_df.corr()
sns.heatmap(corr, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



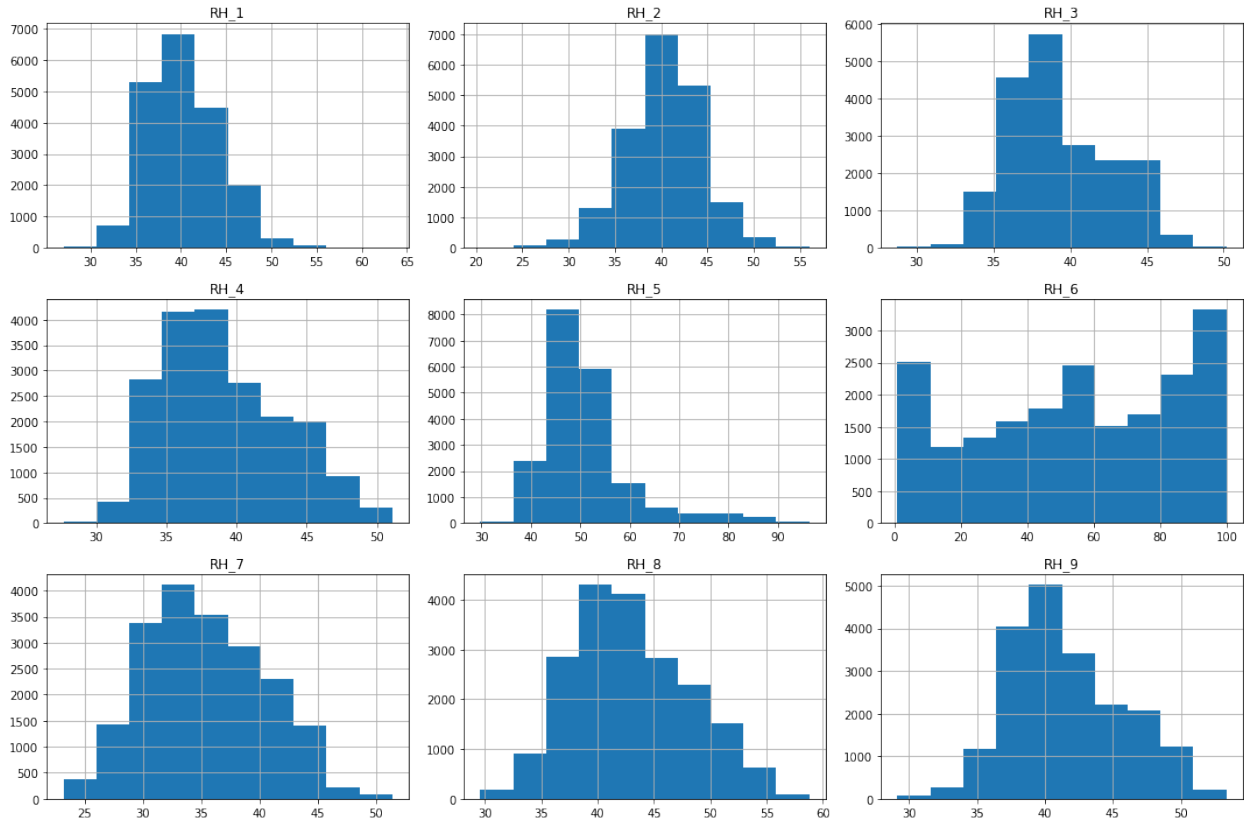
```
plt.figure(figsize=(10,6))
plt.scatter(df["lights"],df["Appliances"],alpha=0.3)
plt.xlabel("Lights")
plt.ylabel("Appliances")
plt.title("Lights vs Energy Usage")
plt.show()
```



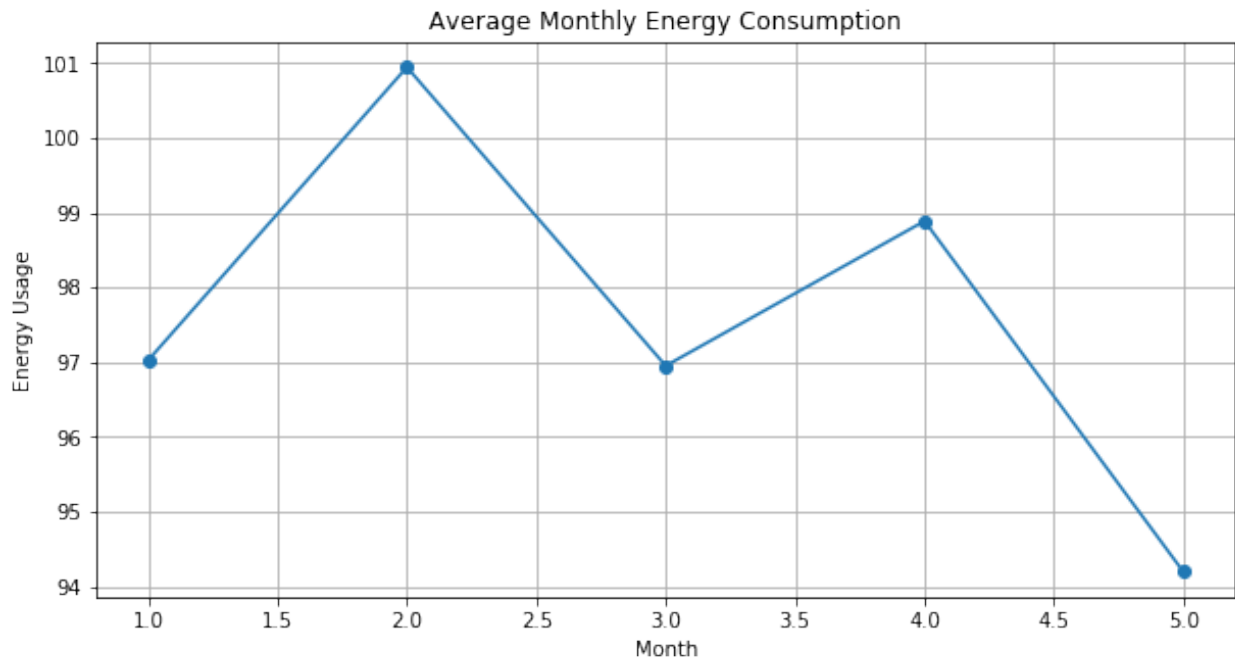
```
temperature_columns=['T1', 'T2', 'T3', 'T4', 'T5', 'T6', 'T7', 'T8', 'T9']  
df[temperature_columns].hist(figsize=(15,10))  
plt.tight_layout()  
plt.show()
```



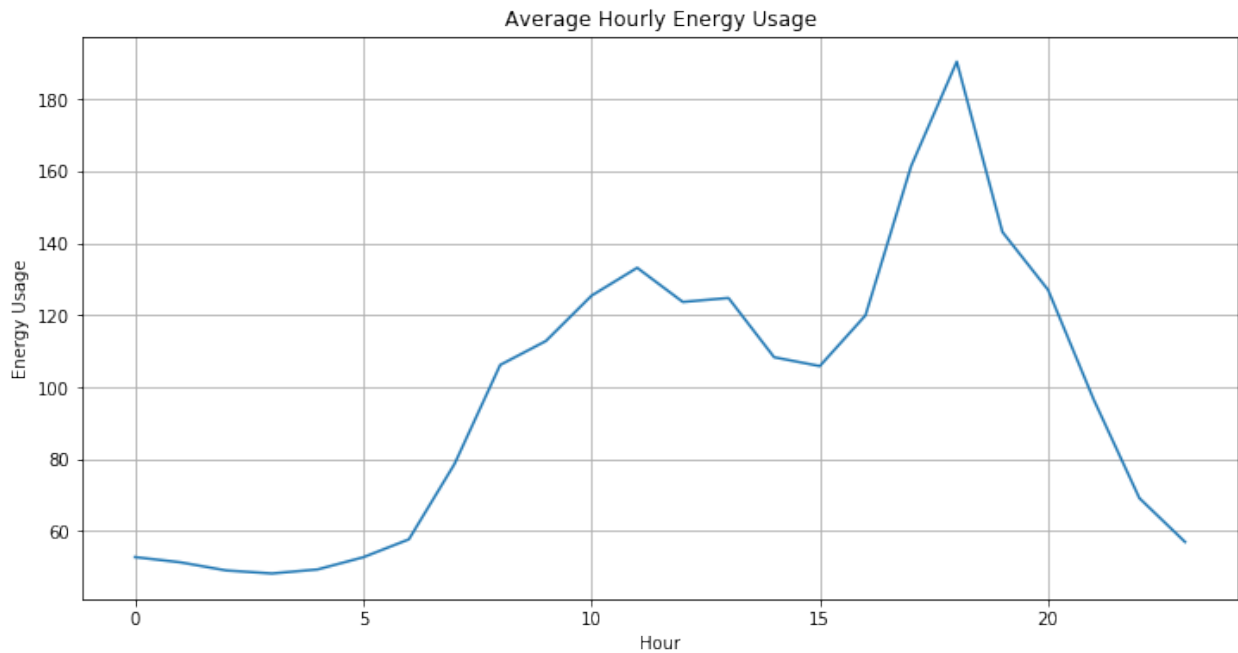
```
humidity_columns=['RH_1','RH_2','RH_3','RH_4','RH_5','RH_6','RH_7','RH_8','RH_9']
df[humidity_columns].hist(figsize=(15,10))
plt.tight_layout()
plt.show()
```



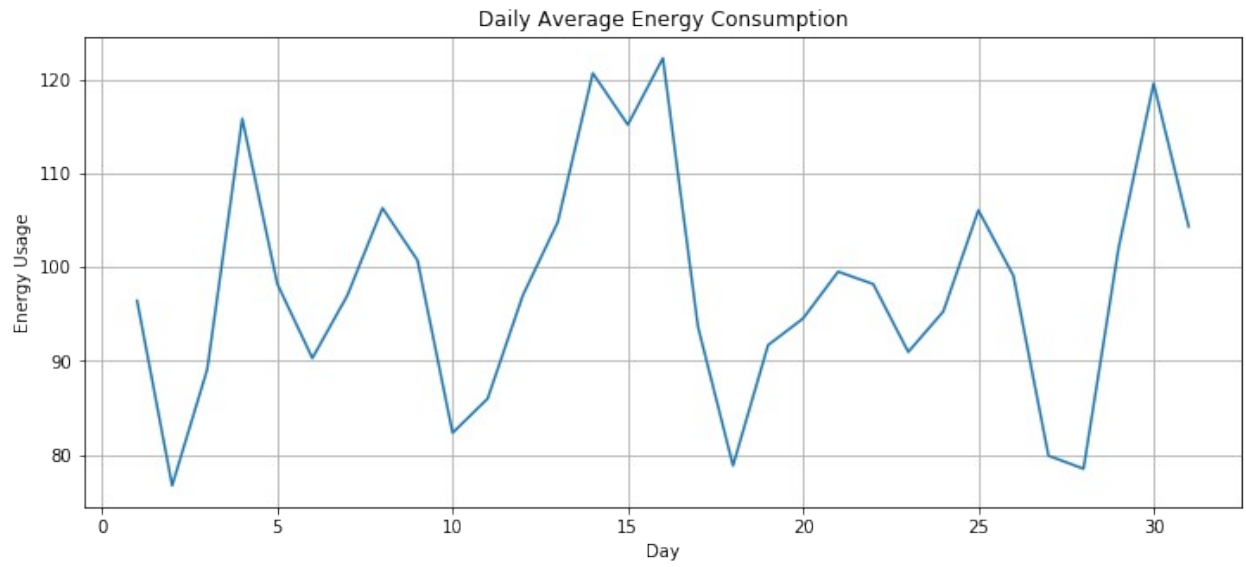
```
monthly=df.groupby("Month")["Appliances"].mean()
plt.figure(figsize=(10,5))
monthly.plot(marker='o')
plt.title("Average Monthly Energy Consumption")
plt.xlabel("Month")
plt.ylabel("Energy Usage")
plt.grid()
plt.show()
```

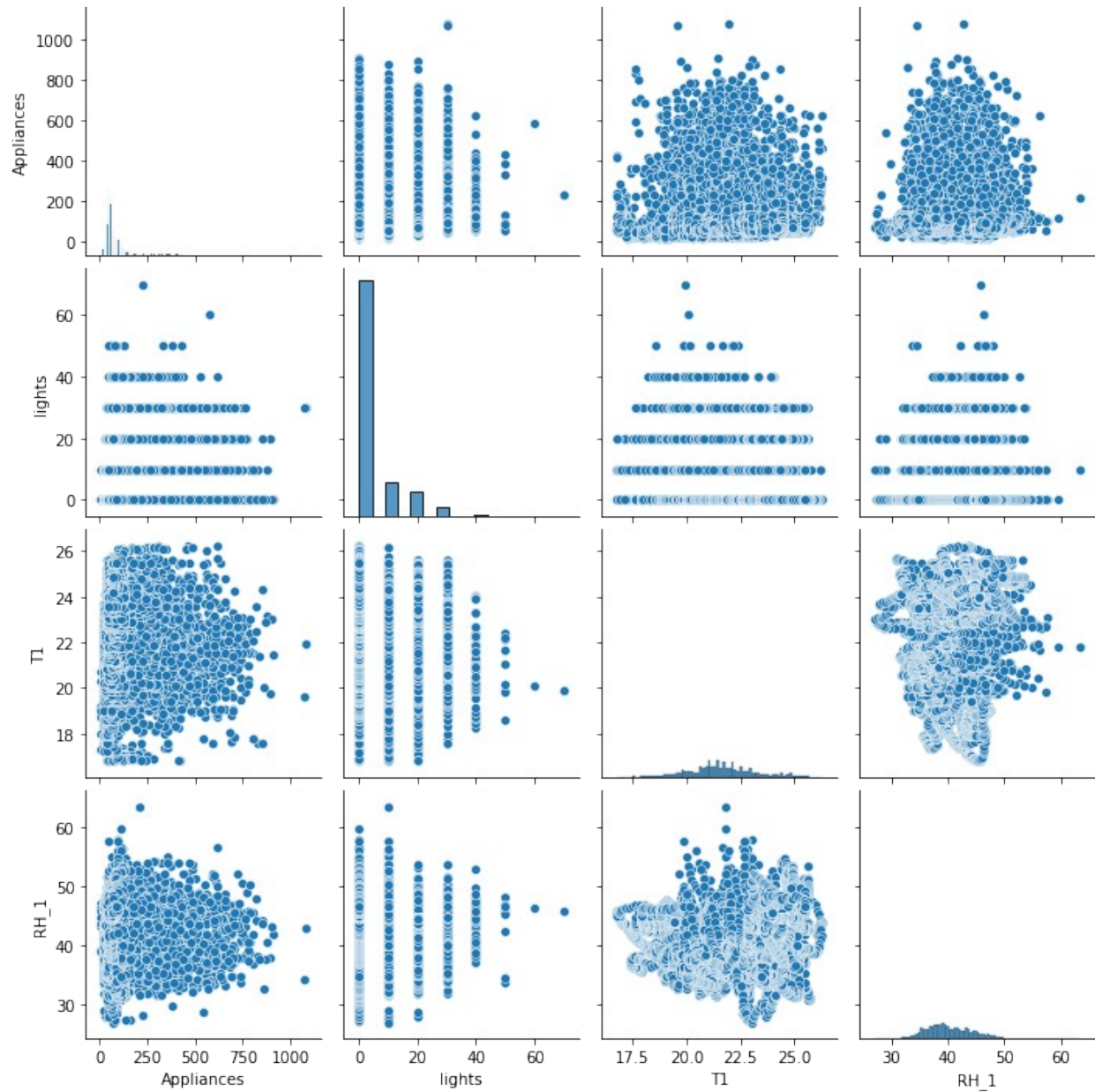
```
hourly=df.groupby("Hour")["Appliances"].mean()  
plt.figure(figsize=(12,6))  
hourly.plot()  
plt.title("Average Hourly Energy Usage")  
plt.xlabel("Hour")  
plt.ylabel("Energy Usage")  
plt.grid()  
plt.show()
```



```
daily=df.groupby("Day")["Appliances"].mean()  
plt.figure(figsize=(12,5))  
daily.plot()  
plt.title("Daily Average Energy Consumption")  
plt.xlabel("Day")  
plt.ylabel("Energy Usage")  
plt.grid()  
plt.show()
```



```
sns.pairplot(df[['Appliances', 'lights', 'T1', 'RH_1']])  
plt.show()
```



```
# Create feature matrix and target variable
```

```
X = df.drop(["Appliances", "date"], axis=1)
```

```
y = df["Appliances"]
```

```
print("Feature Shape :", X.shape)
```

```
print("Target Shape :", y.shape)
```

```
Feature Shape : (19735, 31)
```

```
Target Shape : (19735,)
```

```

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Samples :", X_train.shape[0])
print("Testing Samples :", X_test.shape[0])

Training Samples : 15788
Testing Samples : 3947

from sklearn.linear_model import LinearRegression

lr = LinearRegression()

lr.fit(X_train, y_train)

print("Linear Regression Model Trained Successfully")

Linear Regression Model Trained Successfully

lr_predictions = lr.predict(X_test)

lr_predictions[:10]

array([ 35.7722168 , 252.12988281,  47.3927002 , 113.45227051,
        34.98596191, 200.81097412,  89.30627441, 173.82202148,
        79.35717773, 142.96240234])

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

mae = mean_absolute_error(y_test, lr_predictions)

rmse = np.sqrt(mean_squared_error(y_test, lr_predictions))

r2 = r2_score(y_test, lr_predictions)

print("Linear Regression Performance")
print("-----")
print("MAE :", round(mae,2))
print("RMSE :", round(rmse,2))
print("R2 Score :", round(r2,4))

Linear Regression Performance
-----
MAE : 52.52

```

RMSE : 91.04

R2 Score : 0.1717

```
from sklearn.ensemble import RandomForestRegressor
```

```
rf = RandomForestRegressor(  
    n_estimators=100,  
    random_state=42  
)
```

```
rf.fit(X_train, y_train)
```

```
print("Random Forest Model Trained Successfully")
```

Random Forest Model Trained Successfully

```
rf_predictions = rf.predict(X_test)
```

```
rf_predictions[:10]
```

```
array([ 51.7, 103.3,  45.4,  63. ,  75.2, 123.9, 107.9, 104.5,  51.5,  
       93.3])
```

```
rf_mae = mean_absolute_error(y_test, rf_predictions)
```

```
rf_rmse = np.sqrt(mean_squared_error(y_test, rf_predictions))
```

```
rf_r2 = r2_score(y_test, rf_predictions)
```

```
print("Random Forest Performance")
```

```
print("-----")
```

```
print("MAE :", round(rf_mae,2))
```

```
print("RMSE :", round(rf_rmse,2))
```

```
print("R2 Score :", round(rf_r2,4))
```

Random Forest Performance

MAE : 32.15

RMSE : 67.63

R2 Score : 0.5429

```
comparison = pd.DataFrame({
```

```
    "Model": [  
        "Linear Regression",  
        "Random Forest"  
    ],
```

```
    "MAE": [  
        mae,  
        rf_mae  
    ],
```

```

        "RMSE": [
            rmse,
            rf_rmse
        ],

        "R2 Score": [
            r2,
            rf_r2
        ]
    })

```

comparison

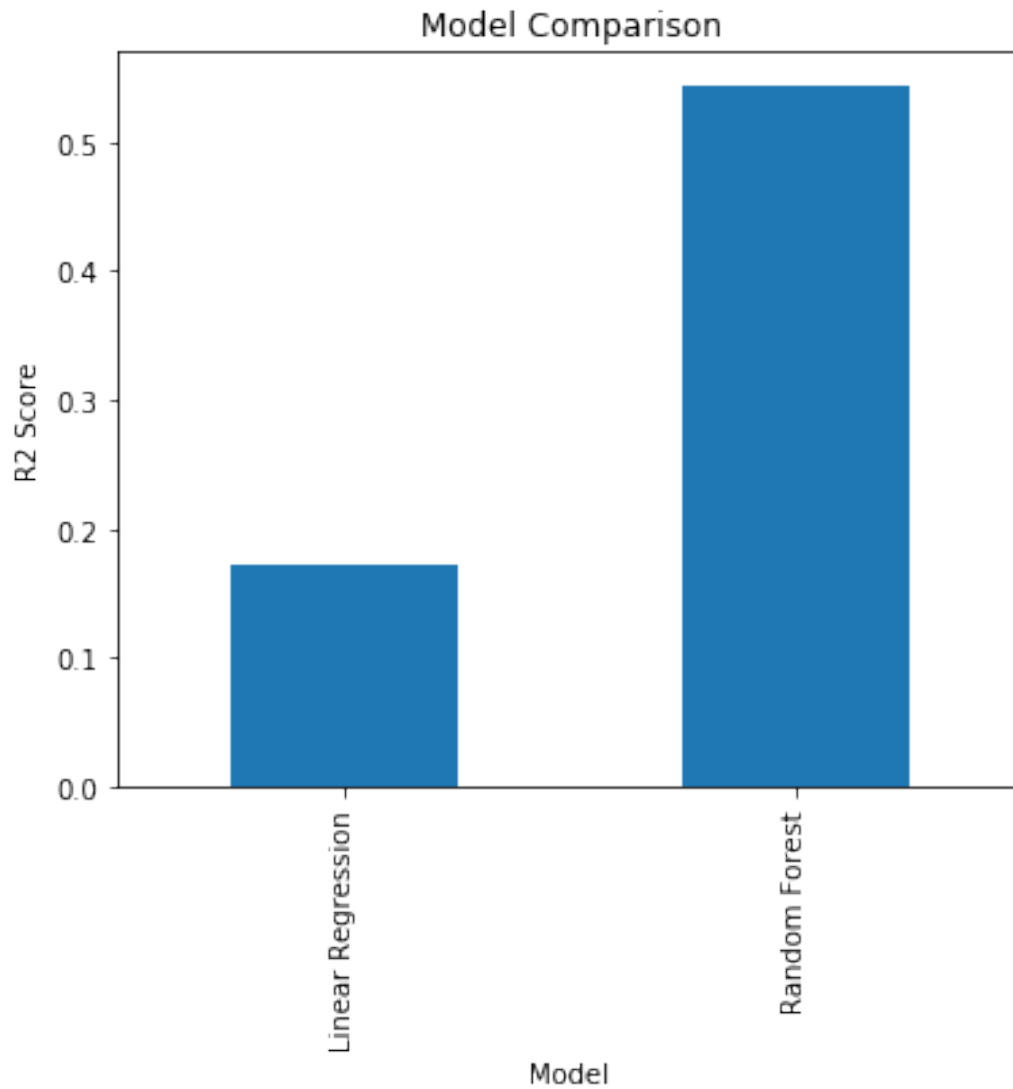
	Model	MAE	RMSE	R2 Score
0	Linear Regression	52.515709	91.041059	0.171740
1	Random Forest	32.148493	67.631558	0.542921

```

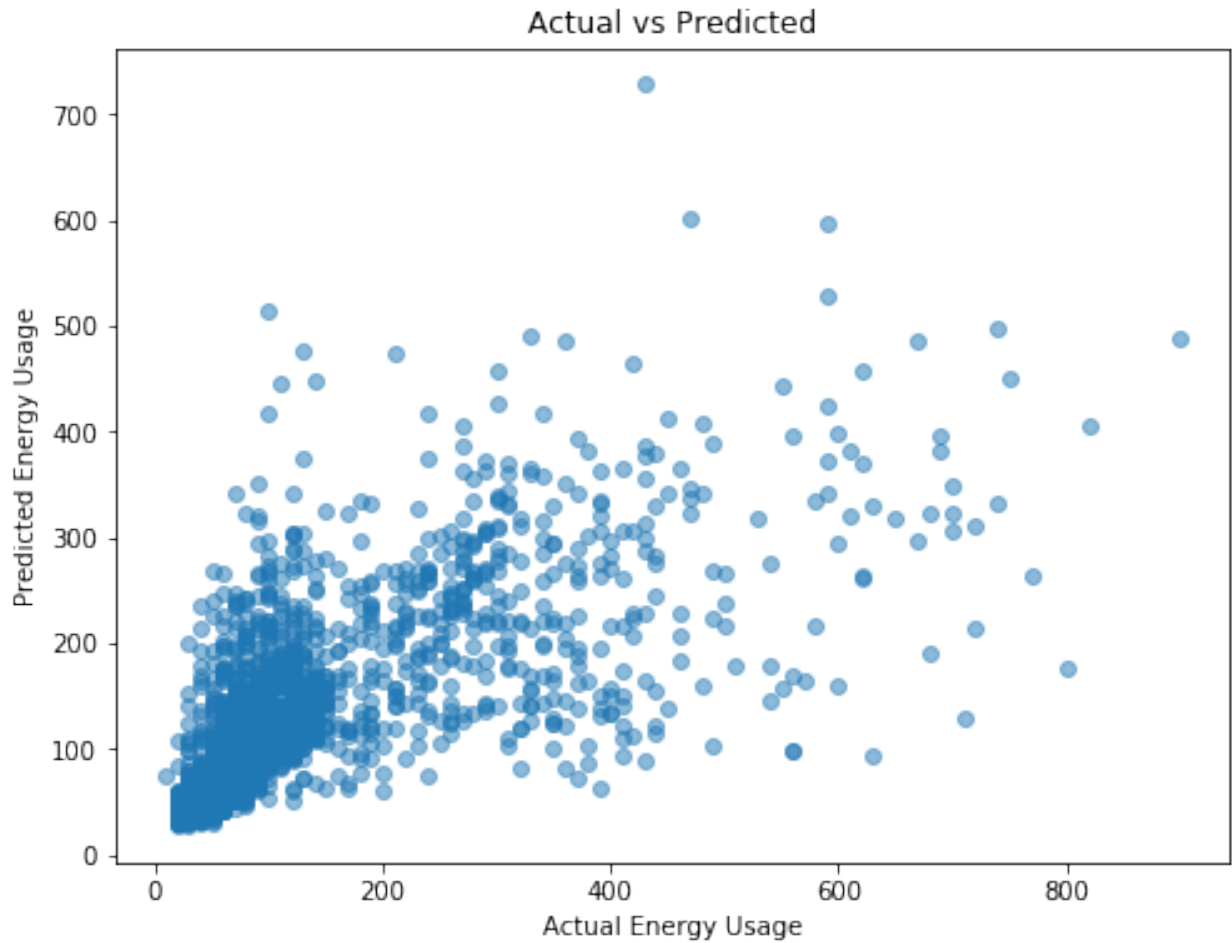
comparison.plot(
    x="Model",
    y="R2 Score",
    kind="bar",
    figsize=(6,5),
    legend=False
)

plt.title("Model Comparison")
plt.ylabel("R2 Score")
plt.show()

```



```
plt.figure(figsize=(8,6))
plt.scatter(y_test, rf_predictions, alpha=0.5)
plt.xlabel("Actual Energy Usage")
plt.ylabel("Predicted Energy Usage")
plt.title("Actual vs Predicted")
plt.show()
```

```
# Select the better-performing model

if rf_r2 > r2:
    best_model = rf
    best_prediction = rf_predictions
    best_model_name = "Random Forest"
else:
    best_model = lr
    best_prediction = lr_predictions
    best_model_name = "Linear Regression"

print("Best Model Selected:", best_model_name)

Best Model Selected: Random Forest

# Assume 0.82 kg CO2 per kWh

carbon_emission = best_prediction * 0.82

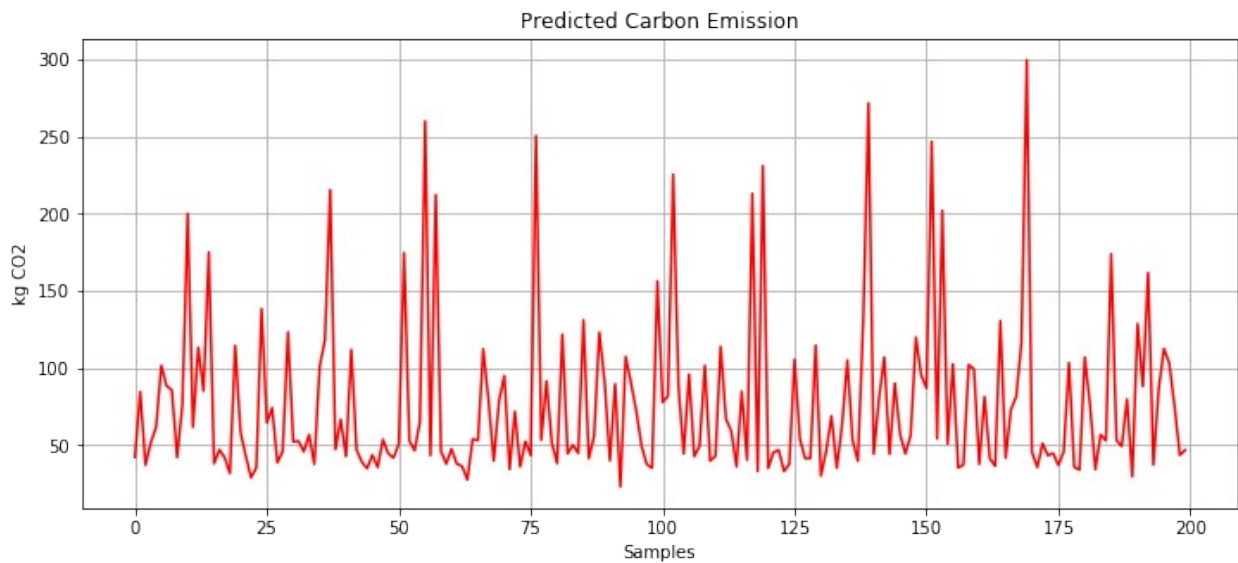
carbon_df = pd.DataFrame({
    "Predicted Energy": best_prediction,
```

```
    "Carbon Emission (kg C02)": carbon_emission
})
```

```
carbon_df.head()
```

	Predicted Energy	Carbon Emission (kg C02)
0	51.7	42.394
1	103.3	84.706
2	45.4	37.228
3	63.0	51.660
4	75.2	61.664

```
plt.figure(figsize=(12,5))
plt.plot(carbon_emission[:200], color="red")
plt.title("Predicted Carbon Emission")
plt.xlabel("Samples")
plt.ylabel("kg C02")
plt.grid()
plt.show()
```



```
average_carbon = carbon_emission.mean()
print("Average Carbon Emission")
print(round(average_carbon,2), "kg C02")
```

Average Carbon Emission
81.09 kg CO2

```
def sustainability_advisor(energy):  
    if energy > 150:  
        return "High Energy Consumption. Use LED lighting, switch off  
unused appliances and consider solar energy."  
    elif energy > 80:  
        return "Moderate Energy Usage. Improve appliance scheduling  
and monitor usage regularly."  
    else:  
        return "Excellent! Energy consumption is sustainable."  
for i in range(10):  
    print("Predicted Energy :", round(best_prediction[i],2))  
    print("Advice :", sustainability_advisor(best_prediction[i]))  
    print("-----")
```

Predicted Energy : 51.7

Advice : Excellent! Energy consumption is sustainable.

Predicted Energy : 103.3

Advice : Moderate Energy Usage. Improve appliance scheduling and
monitor usage regularly.

Predicted Energy : 45.4

Advice : Excellent! Energy consumption is sustainable.

Predicted Energy : 63.0

Advice : Excellent! Energy consumption is sustainable.

Predicted Energy : 75.2

Advice : Excellent! Energy consumption is sustainable.

Predicted Energy : 123.9

Advice : Moderate Energy Usage. Improve appliance scheduling and
monitor usage regularly.

Predicted Energy : 107.9

Advice : Moderate Energy Usage. Improve appliance scheduling and
monitor usage regularly.

Predicted Energy : 104.5
Advice : Moderate Energy Usage. Improve appliance scheduling and monitor usage regularly.

Predicted Energy : 51.5
Advice : Excellent! Energy consumption is sustainable.

Predicted Energy : 93.3
Advice : Moderate Energy Usage. Improve appliance scheduling and monitor usage regularly.

```
def sustainability_score(energy):  
    if energy < 80:  
        return 100  
    elif energy < 120:  
        return 80  
    elif energy < 180:  
        return 60  
    else:  
        return 40  
  
scores = []  
for energy in best_prediction:  
    scores.append(sustainability_score(energy))  
carbon_df["Sustainability Score"] = scores
```

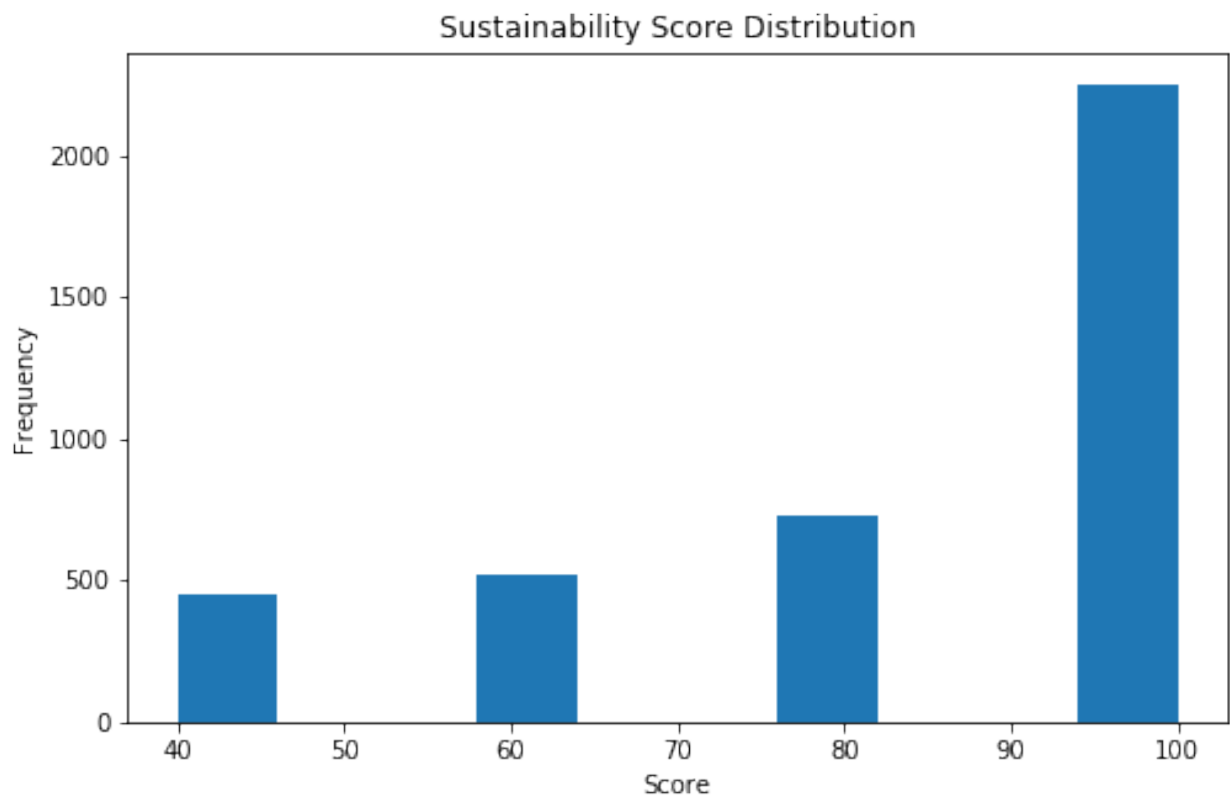
```
carbon_df.head()
```

	Predicted Energy	Carbon Emission (kg CO2)	Sustainability Score
0	51.7	42.394	100
1	103.3	84.706	80
2	45.4	37.228	100
3	63.0	51.660	100
4	75.2	61.664	100

```
plt.figure(figsize=(8,5))
```

```
plt.hist(scores, bins=10)
```

```
plt.title("Sustainability Score Distribution")
plt.xlabel("Score")
plt.ylabel("Frequency")
plt.show()
```



```
import joblib
joblib.dump(best_model, "best_energy_model.pkl")
print("Model Saved Successfully")
Model Saved Successfully
loaded_model = joblib.load("best_energy_model.pkl")
print("Saved Model Loaded")
Saved Model Loaded
sample = X_test.iloc[:5]
prediction = loaded_model.predict(sample)
```

```

prediction
array([ 51.7, 103.3, 45.4, 63. , 75.2])
if best_model_name == "Random Forest":
    importance = pd.DataFrame({
        "Feature": X.columns,
        "Importance": best_model.feature_importances_
    })
    importance = importance.sort_values(
        by="Importance",
        ascending=False
    )
    print(importance.head(10))

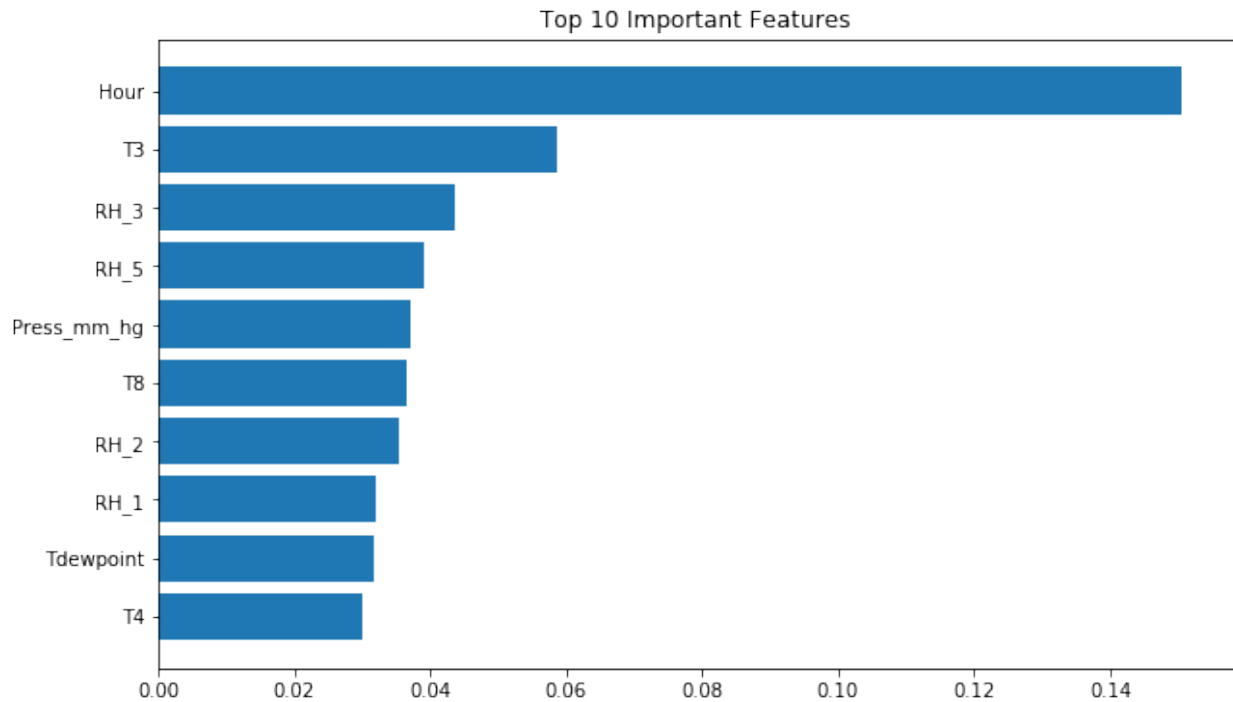
```

	Feature	Importance
30	Hour	0.150570
5	T3	0.058582
6	RH_3	0.043650
10	RH_5	0.039197
20	Press_mm_hg	0.037211
15	T8	0.036640
4	RH_2	0.035372
2	RH_1	0.032130
24	Tdewpoint	0.031853
7	T4	0.030105

```

if best_model_name == "Random Forest":
    plt.figure(figsize=(10,6))
    plt.barh(
        importance["Feature"][:10],
        importance["Importance"][:10]
    )
    plt.title("Top 10 Important Features")
    plt.gca().invert_yaxis()
    plt.show()

```



```
results = pd.DataFrame({
    "Actual": y_test.values,
    "Predicted": best_prediction,
    "Carbon Emission": carbon_emission,
    "Score": scores
})

results.head(10)
```

	Actual	Predicted	Carbon Emission	Score
0	40	51.7	42.394	100
1	90	103.3	84.706	80
2	50	45.4	37.228	100
3	50	63.0	51.660	100
4	70	75.2	61.664	100
5	120	123.9	101.598	60
6	120	107.9	88.478	80
7	70	104.5	85.690	80
8	50	51.5	42.230	100
9	70	93.3	76.506	80

```
results.to_csv("GreenCampusAI_Results.csv", index=False)
print("Results Saved Successfully")
```

Results Saved Successfully

```
print("="*50)
print("GREENCAMPUS AI PROJECT COMPLETED")
print("="*50)
print("Dataset Used :", len(df))
print("Best Model :", best_model_name)
if best_model_name == "Random Forest":
    print("R2 Score :", round(rf_r2,4))
else:
    print("R2 Score :", round(r2,4))
print("Average Carbon :", round(average_carbon,2),"kg C02")
print("="*50)
```

```
=====
GREENCAMPUS AI PROJECT COMPLETED
=====
Dataset Used : 19735
Best Model : Random Forest
R2 Score : 0.5429
Average Carbon : 81.09 kg C02
=====
```